

```
1  "directive">#include <stdio.h>
2  "directive">#include <stdlib.h>
3  "directive">#include <string.h>
4  "directive">#include <unistd.h>
5  "directive">#include <sys/ipc.h>
6  "directive">#include <sys/shm.h>
7
8  "directive">#define SHM_SIZE 1024 "comment">//  Size of the shared memory
    segment
9
10 "keyword">int main() {
11     "keyword">key_t key = "func">ftok("shmfile", 65); "comment">//  Generate a
    unique key for the shared memory segment
12
13     "comment">//  Create a new shared memory segment (or get the identifier of
    an existing one)
14     "keyword">int shmid = "func">shmget(key, "macro">SHM_SIZE, "macro">IPC_CREAT
    | 0666);
15     "keyword">if (shmid == -1) {
16         "func">perror("shmget");
17         "func">exit("macro">EXIT_FAILURE);
18     }
19
20     "comment">//  Attach the shared memory segment to the process address space
21     "keyword">char *shm_ptr = (char*)"func">shmat(shmid, "macro">NULL, 0);
22     "keyword">if (shm_ptr == (char*)(-1)) {
23         "func">perror("shmat");
24         "func">exit("macro">EXIT_FAILURE);
25     }
26
27     "comment">//  Write data to the shared memory
28     "func">strcpy(shm_ptr, "Hello, shared memory!");
29
30     "comment">//  Detach the shared memory segment from the process
31     "keyword">if ("func">shmdt(shm_ptr) == -1) {
32         "func">perror("shmdt");
```

Data written to shared memory: Hello, shared memory!

=== Code Execution Successful ===

```
33     "func">exit("macro">EXIT_FAILURE);
34 }
35
36 "func">printf("Data written to shared memory: Hello, shared memory!\n");
37
38 "comment">// Optional: Remove the shared memory segment
39 "keyword">if ("func">shmctl(shmid, "macro">IPC_RMID, "macro">NULL) == -1) {
40     "func">perror("shmctl");
41     "func">exit("macro">EXIT_FAILURE);
42 }
43
44 "keyword">return 0;
45 }
```